



► **CS476 Final project: Camera-controlled Theremin**

by Abhineet Agarwal and Tommaso Capone

► TABLE OF CONTENTS

01

INTRODUCTION

02

SYSTEM OVERVIEW

03

HARDWARE ARCHITECTURE

04

SOFTWARE ARCHITECTURE

05

SW-ONLY BASELINE

06

RESULTS & CONCLUSIONS

The background is a dark navy blue. It features several abstract geometric elements: a cyan line starting from the left edge, going right, then down to a small circle; a purple line starting from the top center, going down, then right, then up to a series of vertical lines; a white line starting from the top right, going right, then down, then left; a white line starting from the middle right, going right, then down; a white line starting from the bottom right, going right, then up to a small circle; and a cyan arc at the bottom center.

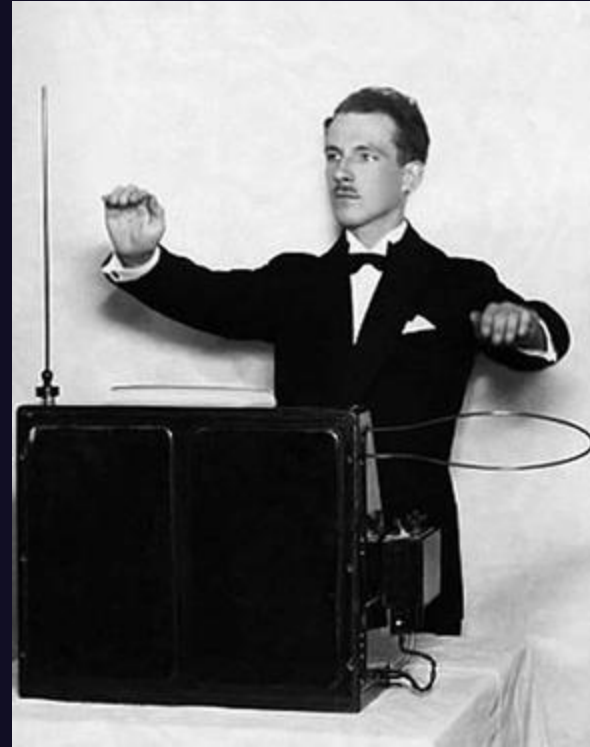
► INTRODUCTION

01

► What is a theremin?

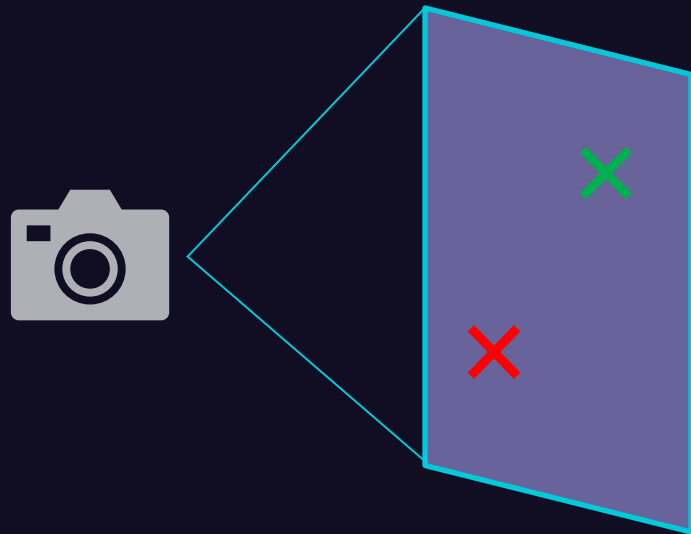
- Electronic musical instrument controlled without physical contact
- Antennas sense the relative positions of the musician's hands
- One hand controls the frequency, the other the volume

Our project aims at implementing a similar system, but digital rather than analog, using the provided camera.



► Our take: A camera-controlled theremin

- The camera replaces the antennas
- Two coloured markers, one per hand, tracked in real time
- Hand vertical position → pitch (one marker) and → volume (the other)
- Implemented on the GECKO5 / ECP5 platform with a 74.25 MHz or 1420 soft CPU



CuteCom - Default

Sessions Help

Close Device: /dev/ttyUSB1 Settings

```
*j
*._
*h
**
}}
*+
**.
```

Input: ** LF Char delay: 1 ms Send file... Plain

```
ff><0xce><0xef><0x8f><0xcf><0xbe><0xde><0x8f><0x9f><0x8c><0x7f><0xe7><0xe3><0xf7><0xf7><0xc6><0xf7><0xbc>w<0xef><0xdd><0xe7>
<0x8d>w<0xcf><0xef><0xce><0xad><0xde><0xfc><0x8d><0xcd><0x9c><0xbf>o<0xd7><0xae><0xdd><0xdf><0xe7><0xce><0xcd><0xee><0xff>
<0xfb><0xcf>c<0xbf>c<0xff><0xf7><0xdf><0xff><0xef><0xd7><0xbe><0xef><0x8f>c<0xeb><0xcf><0xe7><0xce><0xff><0xff><0xfb><0xdd><0xd
7><0xff><0x9d><0xf7><0x8d><0xce><0xd7>k<0xdf><0xc7><0xf6><0xbf><0xdf><0xde><0xef><0xde><0xce><0xcf><0xce><0xdf><0xcf><0xef><
0xad>k<0xe7>CS-476 Embedded System Design
[09:46:44:655] Openisc based virtual Prototype.
[09:46:44:655] Build version: Sun Jul 27 10:19:51 AM CEST 2025
[09:46:45:920]
[09:46:45:920] I am CPU 1 of 1 running at 74.24 MHz.
[09:46:45:920]
[09:46:45:920] Known RS232 commands:
[09:46:45:920] ** Start the program loaded in target
[09:46:45:920] *p Set programming mode (default)
[09:46:45:920] *v Set verification mode
[09:46:45:920] *i Show info on program in target
[09:46:45:920] *t Toggle target between SDRam (default) and Flash
[09:46:45:920] *m Perform simple SDRam memcheck
[09:46:45:920] *s Check SPI-flash chip
[09:46:45:920] *e Erase SPI-flash chip
[09:46:45:920] *r Run program in SPI-flash
[09:46:45:920] *f Store program loaded in SDRAM to SPI-Flash
[09:46:45:920] *c Compare program loaded in SDRAM with SPI-Flash
[09:46:45:920] *h This helpscreen
[09:46:45:920]
```

Clear ☐ Hex output ☐ Logging to: /home/capbros/cutecom.log





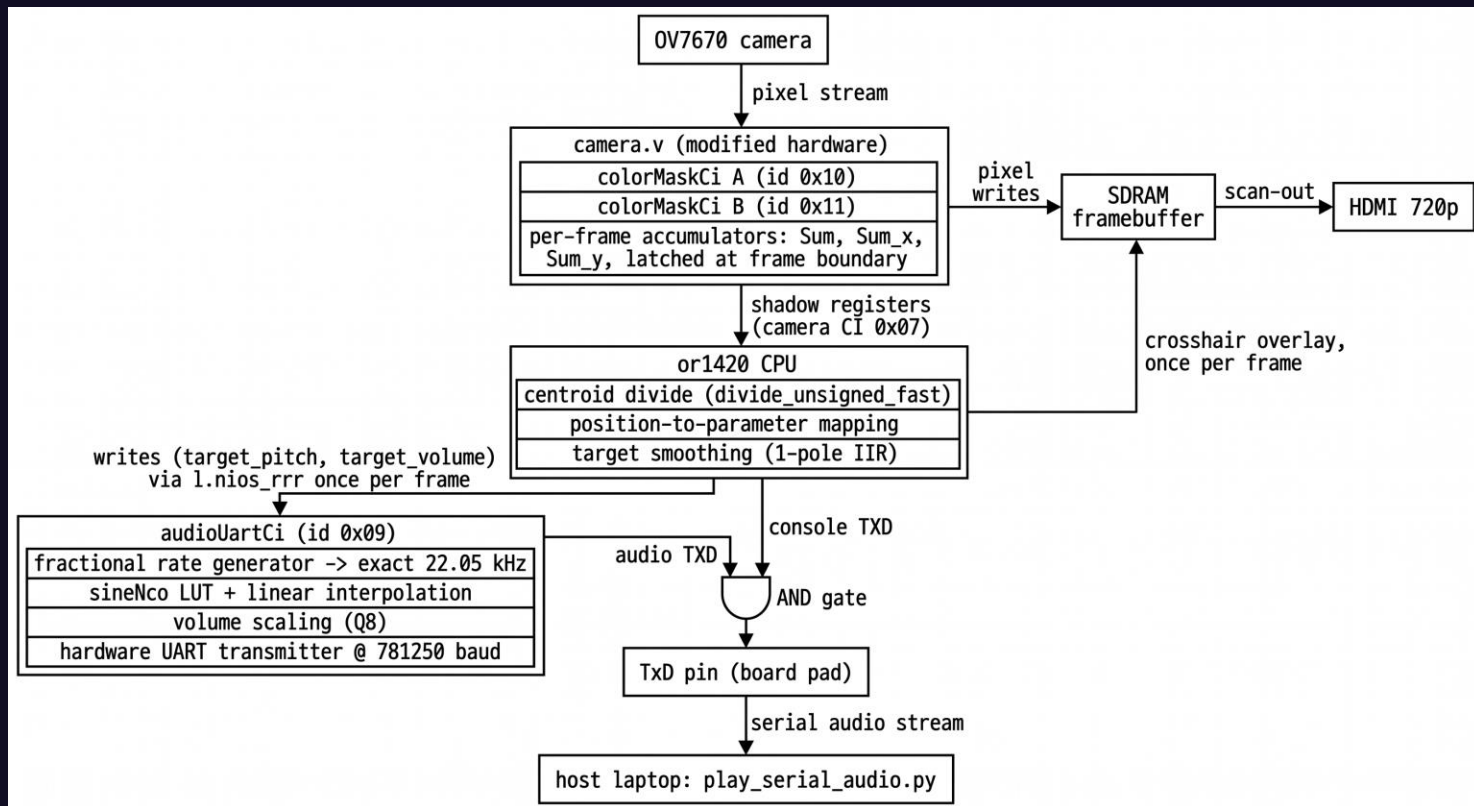
► **SYSTEM OVERVIEW**

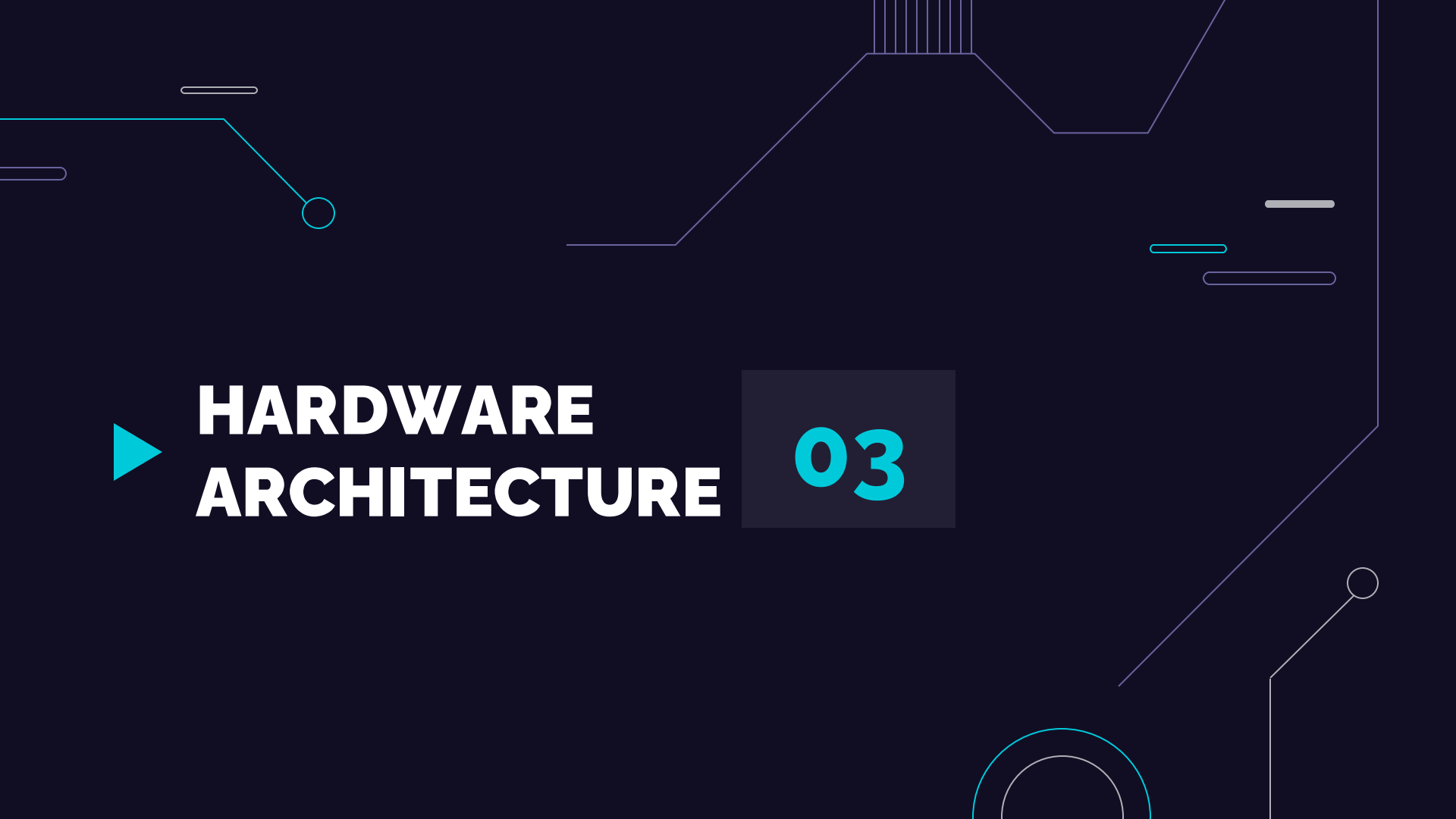
02

► The full pipeline

- The OV7670 camera records the colored markers
- Centroids are computed by filtering pixels based on RGB thresholding and averaging their positions
- The y coordinates are mapped to pitch and volume
- A numerically controlled sine oscillator produces 16-bit audio samples
- Samples are sent over UART
- The host pc plays audio with a python script

Final block diagram



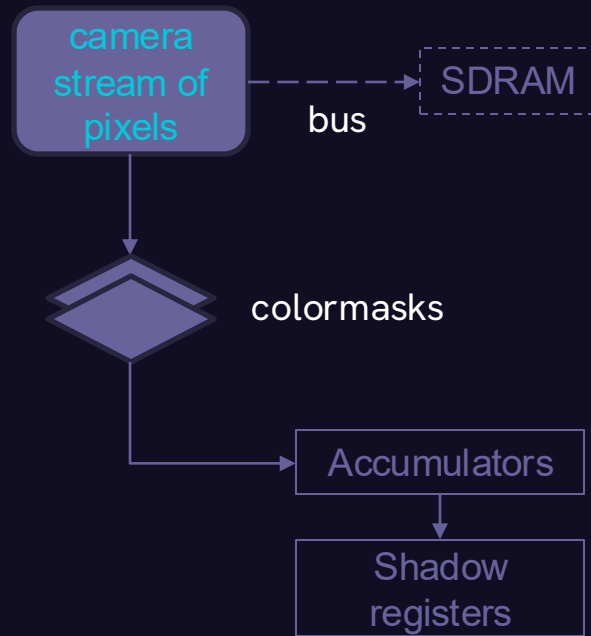
The background features a dark blue field with white and light blue geometric lines. These lines form a circuit-like pattern, including straight segments, right-angle turns, and small circles. Some lines are thicker than others, and there are several horizontal bars of varying lengths. The overall aesthetic is technical and modern.

HARDWARE ARCHITECTURE

03

► Streaming Feature Extractor: modified camera.v

- Pixels arrive from the sensor in raster order
- Two parallel colorMaskCi classifiers per pixel (one per marker)
- Three running sums per marker: Σ_1 (count), Σx , Σy
- Latched into shadow registers at frame boundary
- The CPU never sees a single pixel



► ColorMaskCi: the pixel classifier

- A combinational RGB565 bounding-box test in hardware
- Six parallel comparisons: red, green, and blue each have a low and a high bound
- All six bounds packed into one 32-bit register, programmed via `lnios_rrr`
- Two instances at IDs 0x10 and 0x11 - one per marker
- Thresholds learned at startup by sampling a small calibration patch

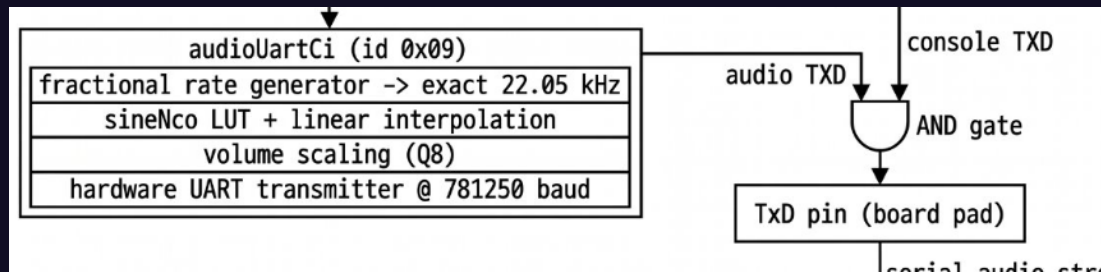
$$(r_{lo} \leq r \leq r_{hi}) \wedge (g_{lo} \leq g \leq g_{hi}) \wedge (b_{lo} \leq b \leq b_{hi})$$

► The architectural pivot: autonomous audio

- First design: a CPU loop emitting one sample every $45\text{ }\mu\text{s}$, paced by a cycle counter
- Wall on real hardware: the SPR holding CPU frequency reads zero $\rightarrow$ the pacing math collapsed
- Insight: audio sample emission is a streaming hardware problem, not a CPU problem
- New design: the CPU just writes (pitch, volume) targets - hardware handles the rest
- The 22.05 kHz deadline now lives entirely inside hardware

► audioUartCi internals

- Exact-rate generator: a Bresenham-style 32-bit fractional accumulator divides 74.25 MHz to 22050 Hz with zero long-term drift
- NCO core: a 256-entry signed-12 sine LUT with linear interpolation between entries
- Volume: a Q8 multiplier produces the final signed-16 sample
- Hardware UART transmitter: 781250 baud, signed-16 little-endian, 20-bit shift register for two back-to-back frames
- AND-mux on TxD: one board pin carries both the audio and debug-console UARTs

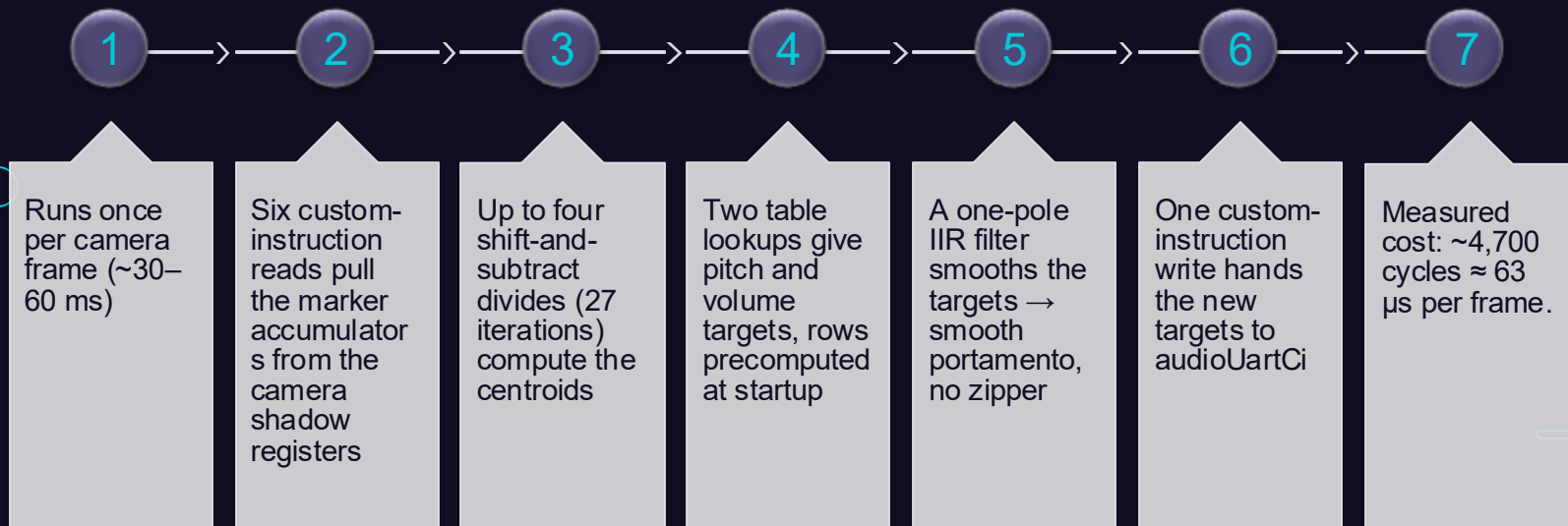




▶ SOFTWARE ARCHITECTURE

04

The per-frame software loop



► **SOFTWARE-ONLY
BASELINE**

05

► The software-only baseline: what changes

- Same calibration, same mapping
- Segmentation runs as a per-pixel C loop, not in hardware
- Audio is emitted by the CPU itself
- Helper functions are bit-identical to the Verilog classifier → identical segmentation results
- 307,200 iterations per frame, with no data cache

```
for (i = 0; i < H; i++)  
  for (j = 0; j < W; j++) {  
    rgb_pixel_t p;  
    extract_packed_rgb(&p, swap_u16(g_frame[i*W + j]));  
    if (check_color_th(&maskA, &p)) { mA.sum++; mA.xsum += j; mA.ysum += i; }  
    if (check_color_th(&maskB, &p)) { mB.sum++; mB.xsum += j; mB.ysum += i; }  
  }
```



▶ **RESULTS & CONCLUSIONS**

06

► Cycle count: accelerated vs software-only

- Measured with profileCi wrapping the per-frame loop body
- Accelerated: mean 4,701 cycles $\approx 63 \mu\text{s}$ per frame
- Software-only: mean 53,278,252 cycles $\approx 718 \text{ ms}$ per frame
- $\sim 11,000\times$ speedup in per-frame CPU work

Path	Cycles / frame	Time / frame	Effective rate
Accelerated (typical)	4,701	$63 \mu\text{s}$	camera-limited (15–30 fps)
SW-only baseline	53,278,252	718 ms	1.4 fps
Ratio (SW / HW)			$\approx 1.1 \times 10^4$

► Resource utilisation

- Synthesised on the ECP5-85 with yosys synth_ecp5 (zero design problems)
- 7,912 LUT4: 9.4% of 84,000
- 4,301 flip-flops: 5.1%
- 20 DSP blocks and 20 block RAMs, roughly 10%

Resource	Used	Capacity (ECP5-85)	Utilisation
LUT4	7,912	≈ 84,000	9.4%
TRELLIS_FF	4,301	≈ 84,000	5.1%
MULT18X18D (DSP)	20	156	12.8%
DP16KD (block RAM)	20	208	9.6%

► Course constraints, all in one table

Course constraint	How we satisfy it
Streaming accelerator	Modified camera.v: per-pixel feature extractor
Self-designed custom instructions	Three total: two colorMaskCi (vision), one audioUartCi (audio)
Hard real-time deadline	22.05 kHz audio rate run entirely by hardware
Software-only baseline	Per-pixel C segmentation, bit-identical logic
Measurable contrast	~11,000× per-frame speedup, profiled with profileCi